



Lab 8: Functions

CSE 4108

Structured Programming I Lab



Lab Tasks

1. toUpper() and beyond!:

We all know about the `toupper()` function from the `<ctype.h>` library, which converts a lowercase letter to an uppercase letter.

i. Write a function `void read_string(char ch[], int length)` that will read the input string.

ii. Write a function `char to_uppercase(char ch)` that will return a character in uppercase letter.

iii. Write a function `void print_string(char ch[], int length)` that will print the whole string in uppercase by repeatedly calling the `to_uppercase()` function on each character.

All these three functions should be used in your main function to take an input string and print the corresponding string in uppercase.

***DON'T** use the built-in `toupper()` function and any string related functions or format specifiers.

2. Find Max, Find Min:

Write a C program to input the value of `n`, and then input `n integers` from the user and find the maximum and minimum of the given numbers using functions. The functions should be named as `Max(int a[])` and `Min(int a[])`.

Sample input:

Enter n: 5

Enter 5 numbers: 9 2 1 -3 13

Sample output:

Max: 13

Min: -3

3. Vowels and Consonants:

Write the following functions. Each of these functions should have a single parameter, which is a c-style string (Character array) as an argument. The function should **only** do what is specified (note that none of these functions do any output to the screen). Your functions should use `const` in the prototype wherever it is appropriate:

1. A function that counts and returns the number of vowels in the character array. (For the purposes of this exercise, we are talking about the standard 5 vowels - A, E, I, O, U).
2. A function that counts and returns the number of consonants in the string. Reuse the previous function written in (1) for this task.

In your main function, you will take a single string as an input, and print the number of vowels and consonants by calling the functions (1) and (2).

***DON'T** use the built-in functions from `ctype.h` or `string.h`.

4. Leap Year!:

Write a C function that checks whether a year is a Leap year or not. A year is a leap year if:

- It is divisible by 4

- Generally, yes, it's a leap year
- Except if it is divisible by 100
 - Then it is not a leap year
- Unless it is also divisible by 400
 - Then it is a leap year after all.

Sample input1:

Enter a year: 2016

Sample output1:

Yes

Sample input2:

Enter a year: 1900

Sample output2:

No

Sample input3:

Enter a year: 3200

Sample output3:

Yes

5. Digital Root:

Write a function named `digitalRoot(int n)` that takes an integer as parameter and returns the digital root of `n`. The Digital Root of a number is the recursive sum of its digits until we get a single digit number.

Sample input:

Enter n: 98765

Sample output:

8

Explanation: $9+8+7+6+5 = 35$. Then $3+5 = 8$